

---

# **Computer Vision Assignment Report**

---

Mohammed Ahmed Eissa

**8067**

Youssef Abdelsamea Youssef

**8114**

## 1) Introduction

Stereo vision aims to estimate depth by computing the disparity between corresponding pixels in a rectified left and right image. Disparity is defined as the horizontal shift between matching pixels and is inversely proportional to scene depth.

In this work, two approaches for stereo matching were implemented:

1. **SAD / SSD**
2. **Dynamic Programming (DP)**

Both methods were tested on sample stereo images to compute and visualize disparities.

---

## 2) Local Block Matching Disparity (SAD / SSD)

### 2.1 Method Overview

For each pixel in the left image, a square window is extracted. This window is matched with windows along the same row in the right image over a range of candidate disparities.

The matching cost is computed using:

- **Sum of Absolute Differences (SAD)**
- **Sum of Squared Differences (SSD)**

The disparity that minimizes the cost is selected.

This is a local method because each pixel is processed independently.

## 2.2 Code Explanation

- *disparity = np.zeros((h, w))*  
*half\_window = window\_size // 2*
    - Stores the disparity map
    - Ensures the window stays centered on the pixel
- 

- *for y in range(half\_window, h-half\_window):*  
*for x in range(half\_window, w-half\_window):*
    - Only valid pixels are processed to avoid image borders.
- 

- *L = left[y-half:y+half+1, x-half:x+half+1]*
    - Extracts the reference window from the left image.
- 

- *for d in range(max\_disp):*  
*x\_right = x - d*
    - Searches only leftward disparities (standard for rectified stereo).
- 

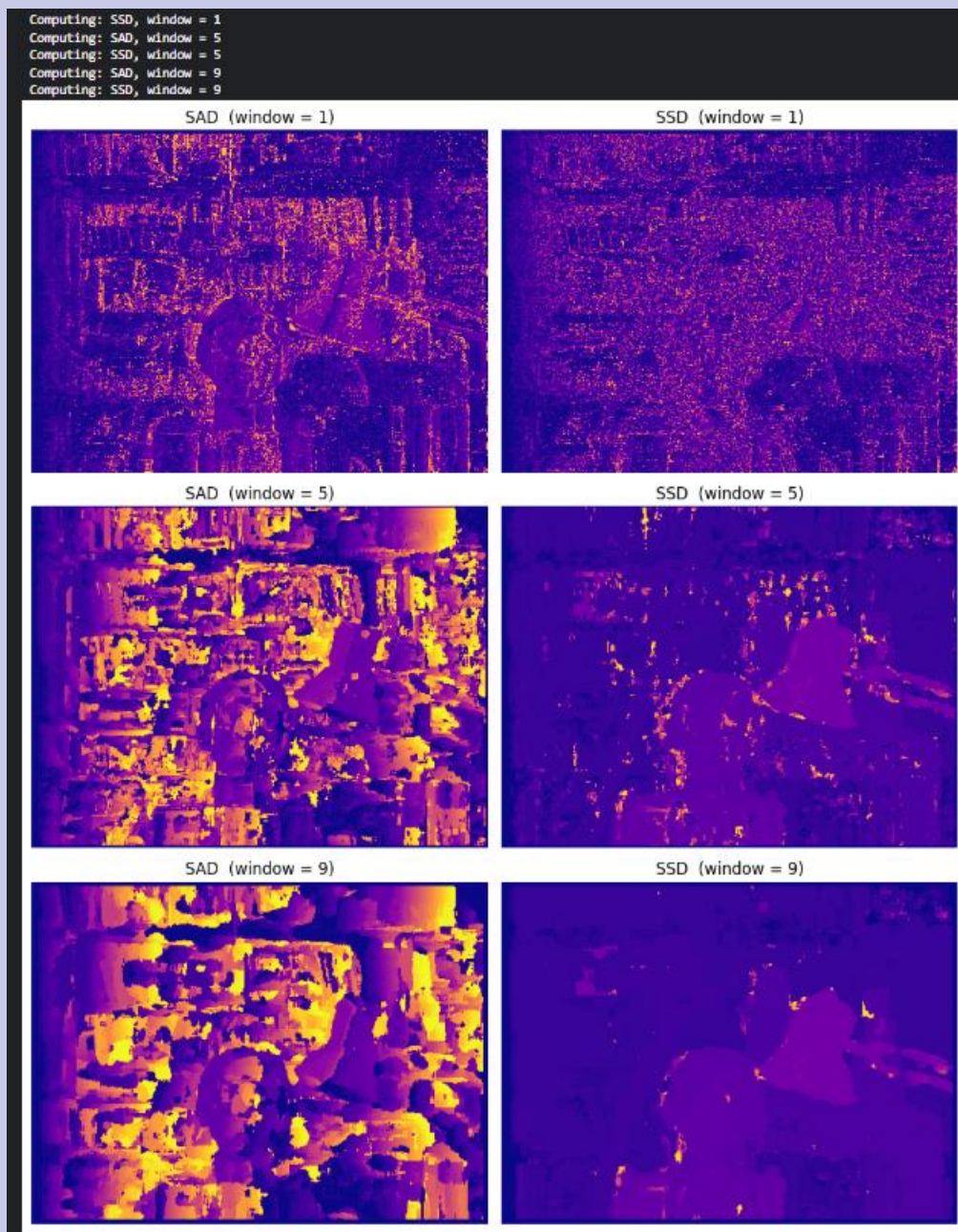
- *R = right[y-half:y+half+1, x\_right-half:x\_right+half+1]*
    - Extracts the corresponding window from the right image.
- 

- *if method == "SAD":*  
*cost = np.sum(np.abs(L - R))*  
*else:*  
*cost = np.sum((L - R) \*\* 2)*
    - SAD is robust and faster
    - SSD penalizes large intensity differences more strongly
- 

The disparity with the minimum cost is stored.

## 2.3 Representative Results

- Produces reasonable disparity maps in textured regions
- Performs poorly near occlusions, depth discontinuities, and low-texture areas
- Sensitive to window size:
  - Small window → noisy disparity
  - Large window → oversmoothing



### 3) Dynamic Programming

#### 3.1 Method Overview

Local methods ignore global consistency and occlusions.

Dynamic Programming solves this by enforcing ordering and smoothness constraints along each scanline.

This approach models stereo matching as a sequence alignment problem.

#### 3.2 Code Explanation

- $dij[i, j] = ((Il[i-1] - Ir[j-1]) ** 2) / (sigma ** 2)$ 
    - Stores the matching cost between pixels i and j.
- 

#### DP Table Initialization

- $D[i, 0] = i * c0$   
 $D[0, j] = j * c0$ 
    - Represents skipping pixels (occlusions).
- 

#### DP Recurrence Relation

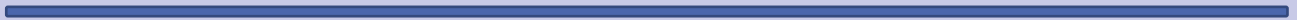
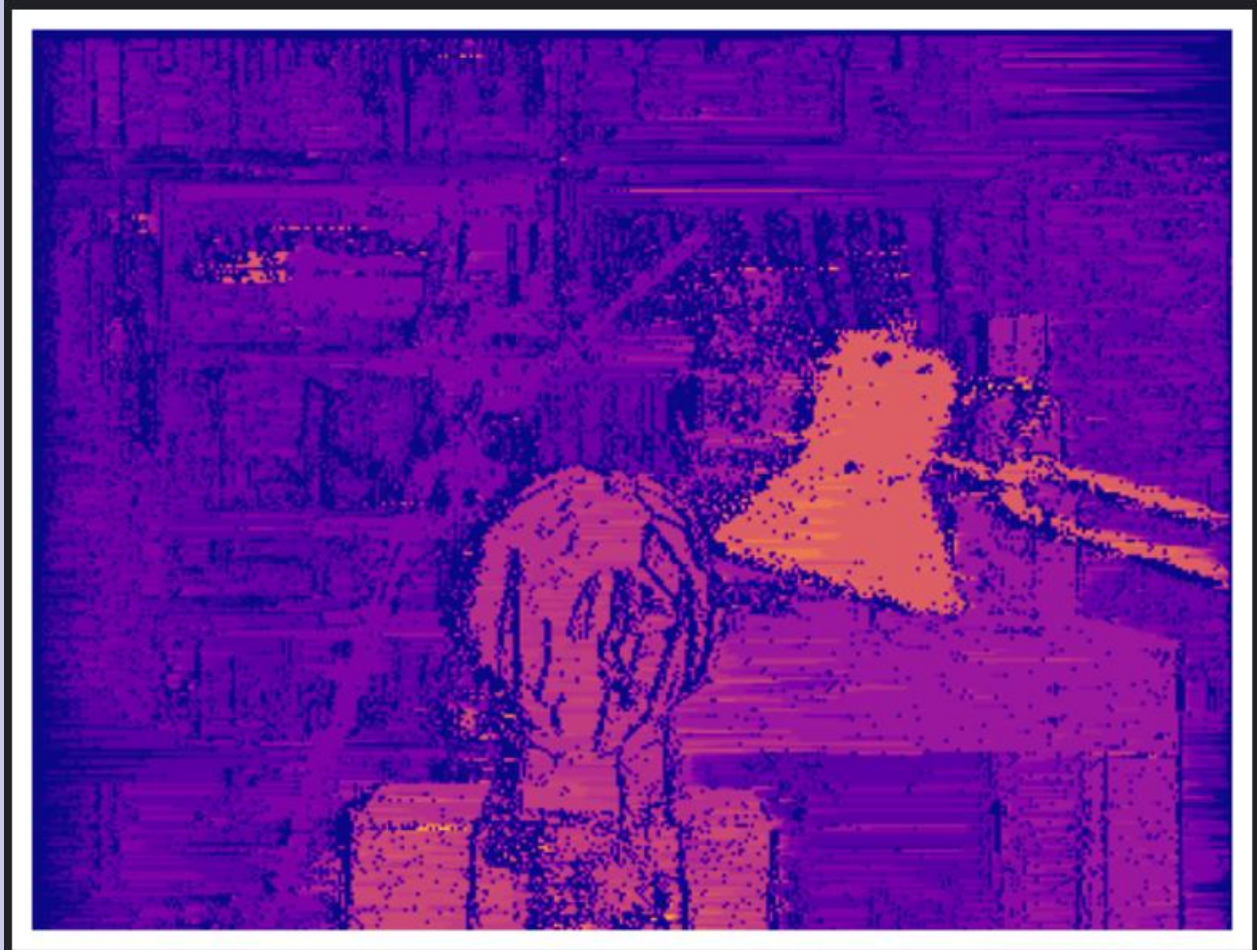
- $match = D[i-1, j-1] + dij[i, j]$   
 $skip\_left = D[i-1, j] + c0$   
 $skip\_right = D[i, j-1] + c0$   
 $D[i, j] = \min(match, skip\_left, skip\_right)$

This corresponds to:

- Match pixel i with j
- Occlude left pixel
- Occlude right pixel

### 3.3 Representative Results

- Handles occlusions explicitly
- Enforces smoothness along scanlines



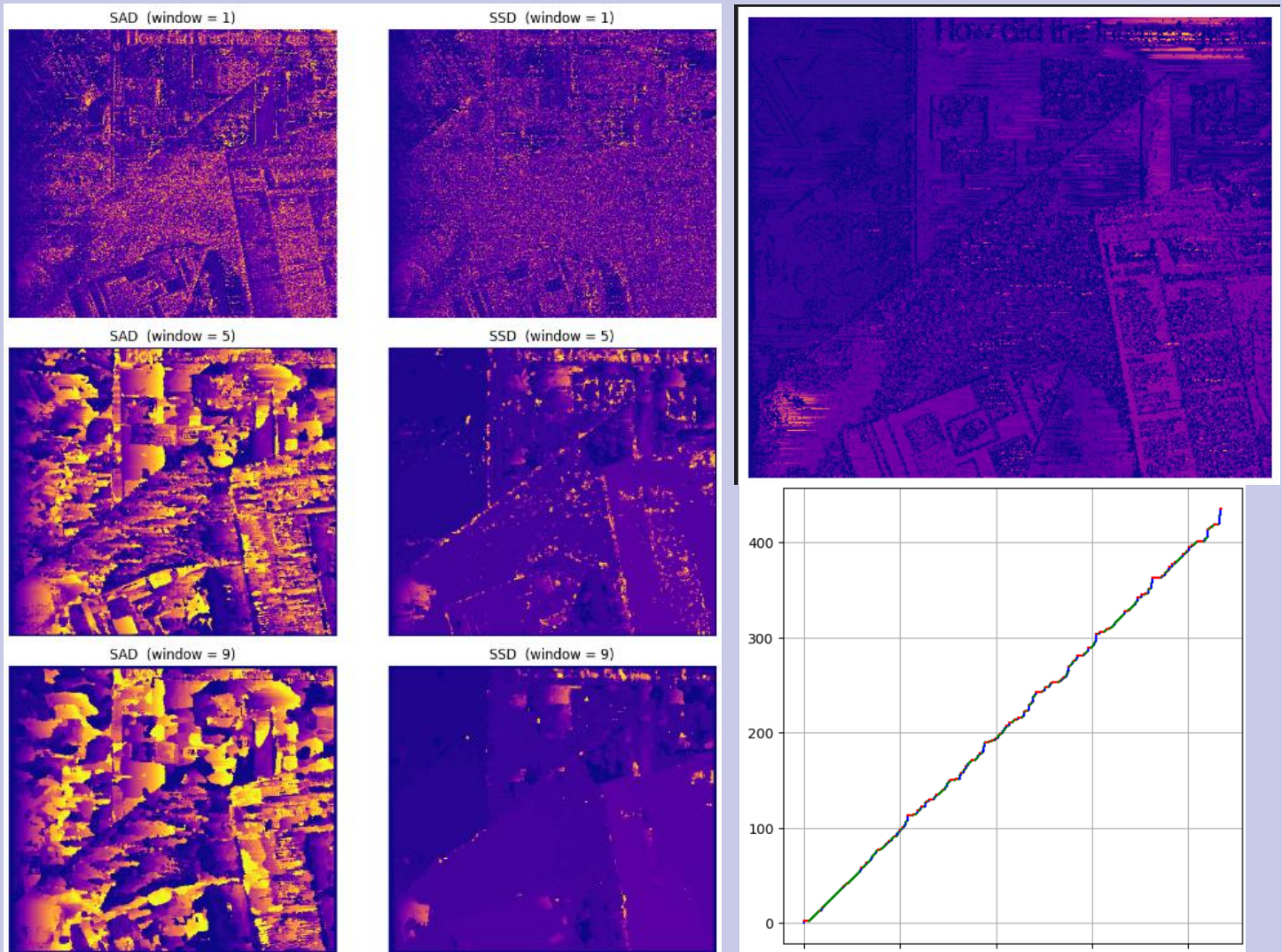


## 4. Experimental Results

### 4.1 Test Images



### 4.2 outputs



---

## 5. Conclusion

Local window-based disparity estimation

Global scanline optimization using Dynamic Programming

Local methods are efficient but unreliable in complex scenes, while DP provides more reliable disparities at the cost of higher computation.

These methods form the foundation for modern stereo algorithms used in depth estimation and 3D reconstruction.